

Neural Networks - Lecture Notes

Course: Introduction to Machine Learning (Lecture 4)

Topic: Feedforward networks, training, and modern architectures

1. What is a neural network?

A neural network is a parameterised function that maps an input vector x to an output vector y through a sequence of layers. Each layer applies a linear transformation followed by a non-linear activation:

$$z = Wx + b$$

$$a = f(z)$$

where W is a weight matrix, b is a bias vector, and f is an element-wise activation function. A network with one hidden layer can approximate any continuous function on a bounded domain (universal approximation theorem), but depth usually buys far more representational power per parameter than width.

The learnable parameters are exactly the entries of every W and b . Training means finding values for these parameters that minimise a loss on the training data while generalising to unseen data.

2. The forward pass

The forward pass evaluates the network from input to output. For a two-layer network:

$$h = f(W_1 x + b_1)$$

$$\hat{y} = \text{softmax}(W_2 h + b_2)$$

The intermediate vectors h are called activations or hidden representations. A useful mental model: each layer learns a new coordinate system in which the task becomes easier - early layers detect local patterns, deeper layers compose them into abstract concepts.

3. Activation functions

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$. Output in $(0,1)$. Saturates for large $|z|$, so gradients vanish; rarely used in hidden layers today.
- Tanh: zero-centred version of sigmoid, still saturating.
- ReLU: $\max(0, z)$. Cheap, sparse, and the default for most feedforward and convolutional networks. Dying ReLU is its main failure mode (neurons stuck at zero).
- Leaky ReLU / GELU / SiLU: smooth or leaky variants that mitigate dead units

and improve optimisation in deep networks.

- Softmax: converts a vector of logits into a probability distribution; used at the output for multi-class classification.

4. Loss functions

Mean squared error is natural for regression:

$$L = (1/n) * \sum_i (y_i - \hat{y}_i)^2$$

Cross-entropy is the standard choice for classification:

$$L = -\sum_i y_i * \log(\hat{y}_i)$$

Cross-entropy measures how surprised the model is by the true label. It pairs with softmax for a numerically stable gradient: the derivative with respect to the logits is simply $(\hat{y} - y)$, which is why classification networks are so well behaved.

5. Backpropagation

Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule backward through the computation graph. For a composition $L = f(g(h(x)))$:

$$dL/dx = (dL/df) * (df/dg) * (dg/dh) * (dh/dx)$$

Each layer stores its activations from the forward pass; the backward pass reuses them to avoid recomputation. The algorithm is exact (not an approximation) and costs roughly the same as two forward passes. Autodiff frameworks (PyTorch, JAX) build the graph for you, but understanding the chain rule is essential for debugging shapes and vanishing gradients.

6. Optimisation

Gradient descent updates parameters in the direction that reduces the loss:

$$\theta \leftarrow \theta - \eta * dL/d\theta$$

- Stochastic gradient descent (SGD) estimates the gradient from a mini-batch. Mini-batches give noisy but unbiased estimates and make GPU parallelism worthwhile.
- Momentum accumulates a velocity term, smoothing noisy gradients and accelerating through ravines.
- Adam combines momentum with per-parameter adaptive learning rates and is a strong default for most tasks.

- Learning rate is the single most important hyperparameter. Too large and the loss diverges; too small and training stalls. Schedules (cosine decay, warm-up) usually help.

7. Regularisation

- Weight decay (L2) penalises large weights and shrinks the effective capacity of the model.
- Dropout randomly zeroes activations during training, forcing redundant and robust representations. Disable it at inference time.
- Early stopping halts training when validation loss stops improving.
- Data augmentation (crops, flips, noise) expands the effective dataset.
- Batch normalisation stabilises activations and often allows higher learning rates.

8. Modern architectures

- Convolutional networks (CNNs) exploit spatial locality with shared filters; pooling reduces resolution. They dominate vision.
- Recurrent networks (RNNs, LSTMs) process sequences with a hidden state, but suffer from vanishing gradients over long spans.
- Transformers replace recurrence with self-attention, allowing every position to attend to every other in parallel. Attention weights are learned similarities; multi-head attention lets the model attend to several relationships at once.

9. Common pitfalls

- Overfitting: training loss falls while validation loss rises. Fix with more data, regularisation, or a smaller model.
- Vanishing / exploding gradients: caused by repeated multiplication of small or large Jacobians. Mitigate with ReLU, residual connections, or gradient clipping.
- Data leakage: information from the validation set sneaks into training (e.g. normalising before splitting). Always split first.
- Imbalanced classes: accuracy is misleading when one class dominates. Use precision, recall, F1, or resample.
- Not shuffling: correlated batches bias the gradient estimate.

10. Key formulas

forward: $a = f(Wx + b)$
 loss (CE): $L = -\sum y * \log(\hat{y})$
 update: $\theta \leftarrow \theta - lr * dL/d\theta$
 chain rule: $dL/dx = (dL/df)(df/dg) \dots (dh/dx)$
 ReLU: $f(z) = \max(0, z)$

softmax: $p_i = e^{z_i} / \sum_j e^{z_j}$

11. Study questions

1. Why does cross-entropy pair so well with softmax?
2. What exactly does the backward pass reuse from the forward pass?
3. Explain dying ReLU and two ways to mitigate it.
4. Contrast SGD, momentum, and Adam in one sentence each.
5. Why is splitting the data before normalisation important?